Programming as Theory Building

Author: *Peter Naur*

Presenter: *Ruijie Xu; Ani Meliksetyan*

About Author

Peter Naur

(Oct 25 1928 - Jan 3 2016)

- Danish computer science pioneer
- 2005 Turing Award winner
- BNF notation contributor
- Language ALGOL 60 creator

Footnotes:

BNF(Backus-Naur-Form) notation:

- A meta-syntax notation used to define the grammar of programming languages, file formats, protocols.

ALGOL 60:

- A seminal procedural, imperative programming language released in 1960 that introduced core concepts of structured programming.

About Author

- Originally trained as **an astronomer** before moving into computing in the late 1950s.
- Emphasized that programming is primarily a **human intellectual activity**, famously arguing in his paper ***Programming as Theory Building*** that writing programs is about **developing understanding** rather than simply producing code.
- Criticized the name **computer science**, arguing that the field should instead be called **"data science."** His reasoning was that the discipline is fundamentally about **the processing, structuring, and understanding of data**, not about computers themselves.

What is the true nature of programming?

Production of code?

Production of documentation?

Production of specifications?

Naur's answer: Programming is **the building of a theory**.

Why is it the building of a theory?

What is this theory about?

“We know more than we can tell.”

The **Philosophical** Perspective:

- Knowledge is **not** limited to explicit **rules** or formal **descriptions**.
- Some knowledge is **tacit, embodied in human understanding**.
- Programming relies heavily on this kind of **tacit understanding**.

The **Programmer’s Theory:**

- Theory is knowing **how** and **why** to act intelligently.
- Understanding how the **program relates to the real-world problems domain**.
- Explaining the **design** decisions behind the system.
- Guiding future **modifications** and **extensions**.

Programming, therefore, is the process of building and maintaining this theory.

CASE STUDY 01

The Compiler *Knowledge Gap*

When documentation fails to transfer tacit design knowledge

ORIGIN

Group A builds Compiler for Language L

Runs on Computer X · Full design theory held internally

TASK

Group B must extend L → L+M for Computer Y

Group A provides: full docs, annotated texts, personal advice

PROBLEM EMERGES

Group B's solutions ignored existing structure

Proposed patches destroyed power & simplicity

RESOLUTION

Personal advice from Group A was decisive

Group B re-framed solutions · Compiler successfully delivered

CORE INSIGHT

*The gap between what **documentation conveys** and what **tacit theory holds** was bridged only through direct personal contact — no written artifact could substitute for lived knowledge.*

1 Documentation Was Insufficient

Full annotation and written design notes couldn't convey the deeper theory — the 'why' behind architectural choices — to an external team.

2 Group A Spotted Problems Instantly

Group A immediately identified structurally unsound solutions and proposed simple, elegant fixes — because they possessed the theory, not just the text.

3 Theory Lives in People, Not Pages

Programmer knowledge is a theory about the real-world activity the program matches — primarily held by the individuals who built it.

Naur, P. (1985) · "Programming as Theory Building"

CASE STUDY 02

The Industrial System

Theory Erosion

When tacit knowledge departs, large systems become unmaintainable

A SYSTEM CONTEXT

Large real-time industrial monitoring system

~200K lines · customised per install · sensors & display devices

B SPECIALIST ROLE

Installation & fault-finding programmers

Full-time for years · direct contact during design · diagnose via ready knowledge

C KNOWLEDGE TRANSFER FAILURE

Documentation alone cannot fill the gap

Receiving programmers encounter repeated difficulties even with full guidance

D ROOT CAUSE

Faults traced to inadequate understanding

Not absent docs — absent theory · cleared only by those who hold the knowledge

200K	3+	∞
LINES OF CODE	PROGRAMMER GROUPS	DOCS ≠ THEORY

KEY OBSERVATION

Even **highly motivated programmers** with full access to documentation were **unable to conceive of additional documentation** that would help — because the missing element is not data, it is **embodied theory**.

Producer Team

Holds original design theory · provides docs & guidance · cannot fully transmit tacit knowledge

Install & Fault Finders

Continuous immersive contact · diagnose via ready knowledge · bridge between theory & deployment

Receiving Programmers

Armed with docs & guidance · regularly encounter failures · theory gap causes repeated difficulty

Large-program adaptation is essentially dependent on a certain kind of knowledge *held by programmers* closely and continuously connected *with the system — not found in any document.*

Naur, P. (1985) · "Programming as Theory Building"

*A program **is not** just code.*

*It is the **theory held by the programmers who understand it.***

Program Life Depends on the Programmer's Theory

Program Life

- A program lives as long as programmers **hold and maintain the theory behind it**.
- The system remains understandable and modifiable.

Program Death

- When the original theory is **lost**, the program effectively "dies".
- Even if the code remains, it becomes **difficult to understand and modify**.

Program Revival

- A program can sometimes be revived when programmers **reconstruct** the underlying theory.
- This usually requires **significant effort and deep investigation**.

Why Modification Is Difficult

Loss of the original theory

- The program's theory mainly exists **in the minds of the original programmers**.
- When these programmers leave, the **underlying understanding of the system disappears**.

Costly reconstruction

- New developers must **reconstruct the program's theory** to make meaningful changes.
- Code and documentation alone are **insufficient to recover the original understanding**.

Structural degradation and rising costs

- Without a clear theory, modifications are often made as local **patches**.
- Over time, these patches **damage the structure of the program**.
- As a result, maintenance becomes **increasingly difficult and costly**.

Implications for the Role of Programmers

- Programmers cannot simply be replaced like production workers.
- Programmers should be seen as responsible professionals whose work involves building and maintaining the theory of the system.

Why is Naur's idea even more relevant today?

- The value of software development lies in the conceptual understanding behind the design, not only in the final artifact.
- AI can generate large amounts of code. It cannot fully capture the underlying theory of a system.
- Programmers are becoming more like architects whose role is to develop, maintain and communicate the underlying theory of a system.

Thank You!

Presenter: *Ruijie Xu; Ani Meliksetyan*